

Listas

102

Estruturas de Dados - ED

- Maneira de organizar dados para facilitar seu acesso.
- Algumas formas são clássicas:
 - Listas (lista, pilha e fila)
 - Arrays (vetores e matrizes)
 - Tuplas (registros)
 - Árvores
- As LPs sempre possuem primitivas para construção dessas EDs
 - Estruturas de dados embutidas ou nativas (*built-in*).
- Outras EDs mais complexas podem ser construídas combinando as EDs clássicas.

103

Estrutura de Dados Abstrata - EDA

- É uma especificação matemática que define uma coleção de dados e uma série de operações sobre ela.
- É abstrata porque não especifica como as operações são feitas mas somente os dados de entrada e o resultado.
- Numa LP, essa coleção de operações é chamada de **interface** ou *API (Application Programming Interface)*.
- Usuários da EDA devem se preocupar com a interface e não com a implementação, que pode mudar com o tempo.
- Implementar uma EDA requer cuidados quanto à correção e a eficiência da mesma.

104

Listas

- São arranjos sequenciais de informações mais simples.
- Caracterizam-se por permitir o acesso eficiente aos seus elementos em ordem sequencial.
- A definição clássica de uma lista como EDA compreende operações
 - Construção de uma lista vazia.
 - Testar se uma dada lista está ou não vazia.
 - Obter o primeiro elemento de uma lista.
 - Adicionar um novo elemento no início ou no final de uma lista.
 - Retirar o elemento do início ou do final de uma lista.
 - Dentre outras.

105

Listas em Python

- A estrutura conhecida como lista (**list**, em inglês) em Python é bem mais geral do que a EDA lista clássica.
- Pode ser vista como uma implementação tanto de listas como de arrays.
 - Além de acesso sequencial, suportam também acesso direto através de índices.
- Listas são variedades de sequências assim como strings e portanto têm APIs semelhantes.
 - Podem ser indexadas e fatiadas (**slices**).
 - Podem ser concatenadas (+) e repetidas (*).

106

Listas em Python

- Apesar da semelhanças, há diferenças importantes entre listas e strings:
 - Sequência genérica vs. sequência de caracteres.
 - Elementos de listas podem ser alterados individualmente mas os de strings, não.
- Listas constituem o tipo de agregação de dados mais versátil e comum da linguagem Python.
 - Podem ser usadas para implementar estruturas de dados mais complexas como matrizes e árvores, por exemplo.

107

Listas: constantes e índices

- Uma constante do tipo lista é escrita entre colchetes com os elementos separados por vírgula. Exemplos:

```
[ ]          # lista vazia
[6, 28]      # lista com 2 elementos
```

- Os elementos de uma lista podem ser de qualquer tipo, inclusive outras listas. Exemplo:

```
lista = [1, 'a', 2+3j, ['ab', 'CD'], [1, 2, 3]]
```

- Os elementos de uma lista podem ser acessados por índices como strings.
 - O primeiro elemento tem índice **0**
 - O último elemento pode ser indicado pelo índice **-1**

108

Listas: constantes e índices

```
>>> lista = [1, 'a', 2+3j, ['ab', 'CD']]
>>> lista[0]
1
>>> lista[2]
(2+3j)
>>> lista[3]
['ab', 'CD']
>>> lista[-1]
['ab', 'CD']
>>> lista[0] = 2
>>> lista
[2, 'a', (2+3j), ['ab', 'CD']]
>>> lista[-1][0]
'ab'
```

109

Listas: concatenação e repetição

- O operador **+** pode ser usado para concatenar e o operador ***** para replicar.

```
>>> lista = [0] * 4           # lista terá quatro 0
>>> lista
[0, 0, 0, 0]
>>> lista = lista + [1] * 3    # concatena três 1
>>> lista
[0, 0, 0, 0, 1, 1, 1]
```

110

Listas: removendo elementos

- O operador **del** pode ser usado para remover itens de uma lista.
Exemplos:

```
>>> lista = [1, 'a', 2+3j, ['ab', 'CD']]
>>> del lista[2]
>>> lista
[1, 'a', ['ab', 'CD']]
>>> del lista[2][1]
>>> lista
[1, 'a', ['ab']]
```

- O que ocorre ao executar o seguinte comando na lista inicial?

```
>>> del lista[0], lista[1]
```

111

Listas: fatias (slices)

- A notação de fatias também pode ser usada, inclusive para atribuição:

```
>>> lista = [1, 'a', 2+3j, ['ab', 'CD']]
>>> lista[1:]
['a', (2+3j), ['ab', 'CD']]
>>> lista[:1]
[1]
>>> lista[1:2]
['a']
>>> lista[0:-1]
[1, 'a', (2+3j)]
```

112

Listas: atribuição a fatias

- A atribuição a uma fatia requer que o valor atribuído seja uma sequência (um iterável, e.g., uma lista ou uma string).
- A atribuição substitui os elementos da fatia pelos da sequência a ser atribuída. Lembre-se, na fatia **A[i:j]**, o **j** é exclusivo. Exemplos:

```
>>> lista = [1, 'a', (2+3j), ['ab', 'CD']]
>>> lista[1:1] = ['z']
>>> lista
[1, 'z', 'a', (2+3j), ['ab', 'CD']]
>>> lista[1:3] = [['x']]
>>> lista
[1, ['x'], (2+3j), ['ab', 'CD']]
```

Pergunta: porque ['z'] não sobrescreve nenhum item e [['x']] sobrescreve dois?

113

Listas: atribuição a fatias

- Mais exemplos:

```
>>> lista = [1, ['x'], (2+3j), ['ab', 'CD']]
>>> lista[1:-1] = [2, 3, 4]
>>> lista
[1, 2, 3, 4, ['ab', 'CD']]
>>> lista[:2] = 'xyz'
>>> lista
['x', 'y', 'z', 3, 4, ['ab', 'CD']]
```

114

Listas: incrementos em fatias

- É possível usar um terceiro número na notação de fatias designando o incremento.
- O default é 1 , ou seja, toma os itens de um em um do menor para o maior índice.
- Pode-se usar qualquer número inteiro diferente de 0:
 - **lista[0:10:2]** - acessa os 10 primeiros itens da lista tomados de 2 em 2.
 - O elemento **lista[10]** não é acessado.
 - **lista[5:0:-1]** acessa os 5 primeiros itens da lista tomados da direita para a esquerda.
 - O elemento **lista[0]** não é acessado.

115

Listas: incrementos em fatias

- Outros exemplos:

```
>>> a = ['a', 2, 3, 'd', 'x', 4, 5, 'b', 7, 8, 9]
>>> a[0:10:2]
['a', 3, 'x', 5, 7]
>>> a[5:0:-1]
[4, 'x', 'd', 3, 2]
>>> a[:3:2]
['a', 3]
>>> a[::-1]
[9, 8, 7, 'b', 5, 4, 'x', 'd', 3, 2, 'a']
```

116

Listas: incrementos em fatias

- Se um incremento de fatia é diferente de 1, uma atribuição à fatia deve ter o mesmo número de elementos:

```
>>> lista = [1, 2, 3, 4, 5]
>>> lista[0::2] = ['x','y','z']
>>> lista
['x', 2, 'y', 4, 'z']
>>> lista[0::2] = [6, 7]
```

Traceback (most recent call last):

```
File "<pyshell#25>", line 1, in <module>
    lista[0::2] = [6, 7]
```

ValueError: attempt to assign sequence of size 2 to extended slice of size 3

117

Inicializando listas

- Não é possível atribuir a uma posição inexistente de uma lista.

```
>>> vetor = []
>>> vetor[0] = 1
Traceback (most recent call last):
  File "<pyshell#27>", line 1, in <module>
    v[0] = 1
IndexError: list assignment index out of range
```

- Se uma lista vai ser usada como um array, isto é, vai conter um número predeterminado de elementos, é conveniente iniciá-la.

```
>>> vetor = [0] * 10
>>> vetor[0] = 3
>>> vetor
[3, 0, 0, 0, 0, 0, 0, 0, 0, 0]
```

118

Usando *None*

- No uso de ED, às vezes é importante preencher uma posição com um valor “inválido”.
- A melhor opção para esse uso é empregar o valor especial **None**.
 - Não faz parte de tipo nenhum
 - É melhor que usar **0** (zero), **[]** ou **""** (string vazia)
- Útil para criar uma lista “vazia” mais com um número conhecido de posições.
- Não mostra os elementos de forma individual.

119

Usando *None*

- Exemplos:

```
>>> lista = [None] * 5
>>> lista
[None, None, None, None, None]
>>> lista[1]                                # Não mostra o valor
>>>
```

120

Operador *in*

- Permite saber se um elemento pertence a uma sequência dada.
- Serve também para strings. Exemplos:

```
>>> lista = [1, 'a', 'bc']
>>> 1 in lista
True
>>> 2 in lista
False
>>> 'b' in lista
False
>>> 'b' in lista[2]
True
>>> 'bc' in 'abcd'
True
```

121

Funções *len*, *min* e *max*

- **len(lista)** retorna o número de elementos da lista.
- **min(lista)** e **max(lista)** retornam o menor e o maior elemento da lista, respectivamente. Exemplos:

```
>>> lista = [2, 3, 1, 4, 9, 6]
>>> len(lista)
6
>>> min(lista)
1
>>> max(lista)
9
>>> max(['a', 'b', 'c'])
'c'
```

122

Funções *min* e *max*

- As funções **min()** e **max()** podem ser usadas também com vários argumentos ao invés de uma lista. Exemplos:

```
>>> min(4, 3, 1, 2)
1
>>> max(3, 4, 5)
5
>>> min('b', 'c', 'a')
'a'
>>> max('b', 'c', 'a')
'c'
```

123

Função *list*

- Usa-se a função `list()` para converter uma string em uma lista.
- É útil pois uma lista pode ser modificada, mas uma string não pode.
- Para fazer a transformação inversa, pode-se usar o método `join` (veremos métodos depois). Exemplos:

```
>>> lista = list('UESPI PHB')
>>> lista
['U', 'E', 'S', 'P', 'I', ' ', 'P', 'H', 'B']
>>> lista[1] = 'xx'
>>> lista
['U', 'xx', 'S', 'P', 'I', ' ', 'P', 'H', 'B']
>>> ''.join(lista)
'UxxSPI PHB'
```

124

Função *range*

- Retorna uma progressão aritmética de inteiros na forma de um iterador que pode ser transformado em uma lista.
- Forma geral:
`range(início, parada, incremento)`
 - **início** (inclusive e opcional) é o primeiro valor a ser gerado (default: 0).
 - **parada** (exclusive) é o limite da progressão. A progressão termina no último valor antes do índice **parada**.
 - **incremento** (opcional) é o passo da progressão (default: 1)

125

Função *range*

- Exemplos:

```
>>> range(3)
range(0, 3)

>>> list(range(3))
[0, 1, 2]

>>> list(range(2, 10, 2))
[2, 4, 6, 8]

>>> list(range(10, 2, -2))
[10, 8, 6, 4]
```

126

Comando *for*

- Permite iterar sobre os elementos de uma lista.
- Forma geral:
for var in lista:
 comandos
 - Os **comandos** são repetidos para cada item da lista.
 - Durante a repetição, **var** possui o valor do item corrente da lista.
- Uma grande utilidade da função **range()** é construir a lista de iteração.

127

Comando *for*

- Exemplo:

```
>>> for i in range(1,7):
...     print(i, end=' ')
...
1 2 3 4 5 6

>>> lista = [1, 2, 'a', 'ab', 5]
>>> for item in lista:
...     print(item, end=' ')
...
1 2 a ab 5
```

128

Comparando listas

- Listas são comparadas lexicograficamente
 - Se duas listas são iguais até os k -ésimos itens, o resultado da comparação depende da comparação entre os $(k+1)$ -ésimos itens.
 - Se alguma das listas tem somente k elementos, então esta é a menor.
 - Duas listas são iguais **sss** têm o mesmo comprimento e todos os elementos de mesma posição são iguais.
- As operações de $<$ ou $>$ não são realizadas entre lista e números, ou entre listas e strings. Operações de $==$ e $!=$ sim.

129

Comparando listas

```
>>> [1, 2] < [2, 3]
True

>>> [1, 2] < [1, 2, 3]
True

>>> [1, 2] != [1, 2]
False

>>> min([[1], [2, 3], [3, 4], []])
[]

>>> max([[1], [2, 3], [3, 4], []])
[3, 4]

>>> min(0, [], "")      # Não pode ser feito
>>> max(0, [], "")      # Não pode ser feito
```

130

Variáveis do tipo list

- Uma variável do tipo lista na verdade contém uma referência para um valor do tipo lista.
 - Atribuir uma variável a outra, cria uma nova referência mas não uma nova lista.
 - Para se criar um novo valor, pode-se usar uma expressão que retorne o valor desejado.
 - Para saber se duas variáveis se referem ao mesmo valor pode-se usar o operador **is**.
 - Forma geral:

var1 is var2

131

Variáveis do tipo **list**

```
>>> a = b = [1, 2, 3]
>>> c = a
>>> d = c[:]
>>> a is b
True
>>> c is b
True
>>> d is c
False
>> a[1] = 5
>>> c
[1, 5, 3]
>>> d
[1, 2, 3]
```

132

A Classe **list**

- Uma lista é na verdade um objeto de uma classe chamada **list**.
 - Não vimos ainda programação OO, mas alguns pontos devem ser destacados.
- Listas possuem métodos que podem ser aplicados a elas.
 - Um método é semelhante a uma função, mas são invocados a partir do objeto que os contém: **objeto.método(args)**
 - Por exemplo, **lista.reverse()** inverte a ordem dos elementos da lista.
 - Para listar todos os métodos de **list**, escreva **help(list)**.

133

Métodos da classe list: **append**

- **append(item)**

- Acrescenta o **item** no fim da lista.
- Ao contrário de tupla e string, a operação sobre lista altera a lista, e não simplesmente retorna uma lista modificada. Exemplo:

```
>>> lista = [1, 2]
>>> lista.append(1)
>>> lista
[1, 2, 1]
>>> lista.append([4, 5])
>>> lista
[1, 2, 1, [4, 5]]
```

134

Métodos da classe list: **count**

- **count(item)**

- Retorna a quantidade de vezes que o **item** aparece na lista.
- Exemplo:

```
>>> lista = [1, 2, 1, [4, 5]]
>>> lista.count(1)
2
```

135

Métodos da classe list: **extend**

- **extend(lista2)**

- Acrescenta os elementos de **lista2** ao final da **lista**.
- **Importante:** Altera a lista ao invés de retornar a lista alterada.
- Exemplo:

```
>>> lista = [1, 2]
>>> lista.extend([3, 4])
>>> lista
[1, 2, 3, 4]
```

136

Métodos da classe list: **index**

- **index(item[, início[, fim]])**

- Retorna o índice da primeira ocorrência de **item** na lista dentro de um intervalo (de **início** até **fim**).
- Um erro ocorre se **item** não consta da lista. Exemplo:

```
>>> lista = [9, 8, 33, 12]
>>> lista.index(33)
2
>>> lista.index(7)
Traceback (most recent call last):
  File "<pyshell#2>", line 1, in <module>
    lista.index(7)
ValueError: 7 is not in list
```

137

Métodos da classe list: **insert**

- **insert(índice, item)**

- Insere na lista o **item** na posição **índice**. Exemplo:

```
>>> lista = [0, 1, 2, 3]
>>> lista.insert(1, 'dois')
>>> lista
[0, 'dois', 1, 2, 3]
```

- Altera a lista ao invés de retornar a lista. Retorna **None**.
- Atribuições a fatias servem para a mesma finalidade mas são menos legíveis.

```
>>> lista = [0, 1, 2, 3]
>>> lista[1:1] = ['dois']
>>> lista
[0, 'dois', 1, 2, 3]
```

138

Métodos da classe list: **pop**

- **pop([índice])**

- Remove da lista o item na posição **índice** e o retorna.
- Se **índice** não for mencionado, é assumido o último item.
Exemplo:

```
>>> lista = [1, 2, 3, 4]
>>> lista.pop()
4
>>> lista
[1, 2, 3]
>>> lista.pop(1)
2
>>> lista
[1, 3]
```

139

Métodos da classe list: **remove**

- **remove(item)**

- Remove da lista o primeiro elemento igual a **item**.
- Se não existe tal **item**, um erro é gerado. Exemplo:

```
>>> lista = ['oi', 'alo', 'ola']
>>> lista.remove('alo')
>>> lista
['oi', 'ola']
>>> lista.remove('oba')
Traceback (most recent call last):
  File "<pyshell#5>", line 1, in <module>
    lista.remove('oba')
ValueError: list.remove(x): x not in list
```

140

Métodos da classe list: **clear**

- **clear()**

- Remove todos os itens da lista e a torna vazia. Exemplo:

```
>>> lista = [1, 2, 3, 4]
>>> lista.clear()
>>> lista
[ ]
```

141

Métodos da classe list: **reverse**

- **reverse()**

- Inverte a ordem dos itens da lista. Exemplo:

```
>>> lista = [1, 2, 3, 4]
>>> lista.reverse()
>>> lista
[4, 3, 2, 1]
```

142

Métodos da classe list: **copy**

- **copy()**

- Retorna uma cópia com os mesmos itens da lista. Exemplo:

```
>>> lista1 = [1, 2, 3, 4]
>>> lista2 = lista1.copy()
>>> lista1[2] = -3
>>> lista1
[1, 2, -3, 4]
>>> lista2
[1, 2, 3, 4]
```

143

Métodos da classe list: **sort**

- **sort([key=None [, reverse=False]])**

- Ordena a lista em ordem ascendente ou decendente.
- Os argumentos são opcionais. Por default, a lista é ordenada em ordem crescente (ascendente).
- Exemplo:

```
>>> lista = [9, 8, 7, 1, 4, 2]
>>> lista.sort()
>>> lista
[1, 2, 4, 7, 8, 9]
```

144

Métodos da classe list: **sort**

- **sort([key=None [, reverse=False]])**

- É possível obter a ordem decrescente passando **True** para o argumento **reverse**. Exemplo:

```
>>> lista = [9, 8, 7, 1, 4, 2]
>>> lista.sort(reverse=True)
>>> lista
[9, 8, 7, 4, 2, 1]
```

- **Importante:** A notação acima permite passar um argumento sem especificar os anteriores, mas a segunda linha poderia ser:

```
>>> lista.sort(None, None, reverse=True)
```

145

Métodos da classe list: **sort**

- `sort( [key=None [, reverse=False]] )`
 - O argumento **key** especifica uma função aplicada a cada item.
 - Se for passada uma função *f*, em vez de ordenar os elementos baseado em seus valores *v*, ordena baseado em *f(v)*.
 - Exemplo:

```
>>> lista = ['abc', 'de', 'fghi']
>>> lista.sort(key=len)
>>> lista
['de', 'abc', 'fghi']
```

146

Matrizes

- Listas podem ser usadas para guardar matrizes. Por exemplo, podemos criar uma matriz-identidade de 3x3 com o código:

```
matriz = []
for i in range(3):
    matriz.append([0]*3)
    matriz[i][i]=1
print(m)
```

- Não é boa uma ideia iniciar uma matriz como abaixo, por que?

```
matriz = [[0]*3]*3
for i in range(3):
    matriz[i][i]=1
print(matriz)
```

147

Matrizes

- Não se recomenda iniciar uma matriz com o comando `m = [[0]*3]*3` porque o Python cria cópias por referência, e não cópias livres, do objeto interno `[0]*3`.
- Todas as três linhas da matriz apontarão para **o mesmo endereço de memória**. Na alteração de um único elemento, a alteração será replicada em todas as linhas simultaneamente.

```
matriz = [[0]*3]*3
for i in range(3):
    matriz[i][i]=1
print(matriz)
```

- O resultado esperado seria: `[[1, 0, 0], [0, 1, 0], [0, 0, 1]]`
- O resultado real será: `[[1, 1, 1], [1, 1, 1], [1, 1, 1]]`

148

Exercícios

- Escreva um programa que intercale os elementos de duas listas `lista1` e `lista2`. Exemplo:
 - Para `lista1 = [1, 2, 3]` e
 - Para `lista2 = ['a','b','c','d','e']`,
 - O programa deve computar a lista: `[1,'a',2,'b',3,'c','d','e']`
- Escreva um programa para computar o produto de duas matrizes `m1` e `m2`.

149

Exercícios

- Escreva um programa para computar o triângulo de Pascal até a linha n , onde n é um valor inteiro positivo informado via entrada.
 - Lembre-se que o item na i -ésima linha e j -ésima coluna do triângulo de Pascal contém o número de combinações de i elementos j a j .
 - O triângulo deve ser posto numa lista onde o i -ésimo elemento é uma lista com a i -ésima linha do triângulo. Exemplo:

Quantidade de linhas: 7

```
[  
[1],  
[1, 1],  
[1, 2, 1],  
[1, 3, 3, 1],  
[1, 4, 6, 4, 1],  
[1, 5, 10, 10, 5, 1],  
[1, 6, 15, 20, 15, 6, 1]]
```